

High Density 8-bit Multiplier Systolic Arrays for FPGA

Martin Langhammer
Intel Corporation
Martin.Langhammer@intel.com

Sergey Gribok
Intel Corporation
Sergey.Gribok@intel.com

Gregg Baeckler
Intel Corporation
Gregg.Baeckler@intel.com

Abstract —Artificial Intelligence (AI) has become the fastest growing application area for FPGAs. Two types of numerics are needed. Training typically uses floating point arithmetic (which is now widely available as embedded functions in current FPGAs). Inference is typically calculated with lower precision integer numbers, which can be implemented with embedded functions, soft logic, or a combination of the two. INT8 performance is therefore used as a typical benchmarking metric for current FPGAs. Recent publications based on Xilinx devices show the extraction of two INT8 multipliers from a 24x18 multiplier. A paper from Intel describes how to obtain two INT8 multipliers from a 18x18 multiplier, with the help of a small amount of soft logic. In this paper we introduce a number of new INT8 multiplier techniques, starting with the Intel 18x18 multiplier approach. Using both memory and logic resources - for a more balanced use of the FPGA features - we improve the INT8 density, and also show a signed-magnitude (SM) 1.7 construct that is even smaller. To demonstrate the usability of these new multipliers, we develop a scalable systolic array, that contains up to 32,768 SM1.7 multipliers, or 28,800 INT8 multipliers, fit in an Intel Stratix 10 2800 device. Finally, we implement a system architecture that includes input and output flow buffering and control, which can be instantiated directly into a larger AI design, or can enable the FPGA to be used as a standalone accelerator. This system exceeds 400 MHz for the largest array on a mid-speed device (26 TOPS INT8), and can operate up to 600 MHz for smaller array sizes.

I. INTRODUCTION

In deep learning (DL), multiplier density and performance - whether TOPs or TFLOPs - sets the performance expectation of the implementation. Dot product or matrix-vector arrays are the most common structure for these [1]. One advantage of the FPGA is flexibility; dataflow can be configured for each application, and non-linear activation functions like *tanh* and *sigmoid* [2] can be inserted anywhere. Support of simpler functions such as RELU are trivial. Additional optimizations like precision or numerical representation scaling can be varied from design to design. But the FPGA has less raw performance potential compared to ASIC at similar process nodes. Newer FPGA devices are addressing this gap with more embedded features, especially for lower precision floating point and integer numbers. Xilinx has introduced the ACAP concept [3], with embedded processors supporting IEEE754 FP32, INT16, and INT8 representations. The Intel Agilex FPGA [4] now has IEEE754 FP16, BFLOAT16, as well as INT9 support in the DSP Blocks. These two device families have just been introduced, which means that the mainstream devices

generally available still lack the performance required for this important application area.

We also considered different types of numerical representation for this work: signed magnitude is potentially more efficient, and aligns with the precision component of the IEEE754 floating point numerics. In [5], custom floating point numbers with a variable SM mantissa were used, so we have precedence and results on FPGA for this approach. A recent work [6] using a 9 bit custom floating point format on a Xilinx VUP device also uses a signed magnitude mantissa representation, with two mantissa multipliers mapped to a single DSP48. Although not using signed magnitude, [7] points to floating point processing, particularly with shared exponents, as an increasing use model for FPGA. Our goal in this work was not to benchmark another deep learning application, but to improve the performance density of the mainstream FPGA considerably, in order to maintain competitiveness with GPUs and the emerging ASSPs. We took some inspiration from the Microsoft Brainwave work [8][9], where a Stratix 10 2800 device was packed to 92% logic density. According to [10], 80% of the device logic was used by the datapaths, packed to 97%, and the remaining 20% of the logic was control and data distribution, packed to 80% efficiency. Other important points from [9] was that around 2/3rds of the memory blocks were used - presumably largely for data distribution, i.e. feeding the dot products. In this work, we will aim for a 80% device logic resource utilization for our datapaths, while at the same time incorporating some of the free memory into the arithmetic calculations. We will still use all of the DSP blocks that we are able to access. Our contributions are as follows:

- 1) We develop an improvement to published INT8 results to support SM1.7, using a combination of DSP blocks, embedded memory blocks, and soft logic.
- 2) We describe dot structures for SM1.7, that support both high density and high packing efficiency.
- 3) The SM1.7 logic and memory implementation methods are mapped back to the INT8 datapaths, with similar improvements in multiplier density
- 4) We demonstrate a scalable 2D systolic array that fits to deterministic area and geometry automatically - without floorplanning - from a small grid to a full chip design.
- 5) We provide a lightweight, low cost, scalable interface which can source and sink any size of the 2D systolic

arrays to the pins, or a user design, while maintaining the performance of the systolic array

II. PREVIOUS WORK

The production Brainwave deployment [8] used 91% of the logic, 69% of the M20Ks, and 91% of the DSP on the target Stratix 10 2800 device. Much of the M20K resource was used to store activations and weights locally. We will attempt to employ some of the 30% of the unused M20K to perform arithmetic operations in the dot product structure. The clock rate of the Brainwave design was limited to 250 MHz [8], although they earlier reported 300 MHz for their prototypes [9]. Brainwave uses 96K multipliers with a block floating point $\{1.5.2\}$ format per device. We are instead using a SM1.7 format for the mantissa portion. A SM1.7 bit multiplier is much larger arithmetically than the SM1.2 multiplier in Brainwave - by over 10x. The number of SM1.7 multipliers that can be implemented in the FPGA will therefore be significantly less, but we will show that are new techniques will be able to dramatically close this gap in operations.

The Xilinx whitepaper WP486 [11] shows how to extract two INT8 multipliers from a DSP48 if one of the inputs is shared. A reference point in this paper will be the Xilinx VU9P with 6840 DSP48 blocks, giving a maximum possible of 13680 INT8 multipliers using their method. Xilinx does not show any system level results in their whitepaper, although a number of other works based on this have been published recently.

In [12], the Xilinx Supertile is used at double the clock frequency of the surrounding logic (720MHz/360Mhz), but only 55% of the DSP Blocks are used. Although not explicitly stated, this is because the application implementation is memory bandwidth limited, as all of the large memory blocks (URAMs) are used (only 40% of the smaller 36kb memory are used, however). The earlier Xilinx Supertile paper [13] describes the implementation of a large 96x16 (1536 DSP48 blocks) processing array, albeit on the smaller VU3P device. This design takes up 67% of the available DSP48 blocks, and runs at the 775MHz-891MHz (speed grade dependent) datasheet speed. In [12] speed degradation to 92% of this rate is observed in the larger VU9P (which is essentially three VU3P die stitched together). According to [13], 100% DSP use for the Supertile uses only 25% of the device logic, although this 100% is only an extrapolated number based on the realized smaller design. The Supertile paper also uses INT16, while [12] implements INT8 via the technique from [11]. Extrapolating to a maximum systolic INT8 array use on a VU9P (100% DSP and 25% logic utilization) gives 19.7 TOPs. The performance based on the demonstrated 96x16 array replicated thrice in the VU9P is 13.3 TOPs. This array architecture is based on groups of small cascaded DSP Blocks.

In [14], an alternate assembly of the DSP48 Blocks is presented. As opposed to the Supertile, the memory and DSP Blocks operate at the same clock rate. Logic use is reduced - presumably as the data rate matching is not required - but the clock rate is slightly reduced to 650MHz, mapped to a large VU37P device. This design incorporates a more balanced

use of the embedded resources, and uses 100% URAM, 95% DSP, and 40% BRAM. The INT8 extraction method [11] is again used with the DSP Blocks. Several DL applications are benchmarked; although significantly more DSP Blocks are utilized, throughput is reduced by 30% over [12], however latency is reduced by 7X. Assuming that this design scales to the somewhat smaller VU9P, and applying the utilization ratios and clock frequency we calculate a scaled performance of 16.9 TOPs.

In [15] a Stratix 10 2800 device is filled with dot products, arranged as tree structures. A new type of INT8 extraction to obtain two INT8 multipliers per 18x18 DSP Block multiplier (with the use of some soft logic) is described, with an average cost of about 8 ALMs per INT8 multiplier. The INT8 multipliers are grouped in 32 element dot products, and a full chip dot product array is implemented. Fractal Synthesis [10] is used to pack the logic more efficiently, leaving the unused logic and routing available in large contiguous blocks, where it can presumably easily be accessed for any application level functionality. The dot products are individually instantiated, so that all inputs and outputs must be individually sourced and terminated. This is done by using virtual pins. The presented design contains 22,400 INT8 multipliers (700 dot products); with shared inputs this amounts to 283,500 virtual pins - this alone requires 141,500 ALMs of the placed and routed design. The logic for these virtual pins would also be freed up if the connections were from a system level design - as these virtual pins represent 15% of the available logic on the device, this is almost enough to implement our entire Brainwave inspired design, which requires about 20% of the 2800 logic. (The connections to the dot products would largely come from the M20K memories, and not through logic for this design). The Stratix 10 2800 (933K ALMs, 5760 DSP Blocks) device is slightly smaller than the VU9P device (1182 6LUTs, 6840 DSP48s).

Comparing the two Virtex Ultrascale (VUP) designs, we can see that the dataflow architecture has a significant impact on metrics such as throughput and latency, and that the performance level of the final design cannot be independent determined simply by looking at memory size, bandwidth, and number of TOPs. The performance, however, cannot exceed the maximum number of TOPs. The Xilinx targeted designs also rely on the hard cascade feature of the DSP48 blocks, which will introduce a linear latency to any vector operation. In contrast, Brainwave uses a dot product implementation, with a combination of DSP Blocks and soft logic. We note that Microsoft Brainwave is the highest profile production design currently in use, with tens of thousands of nodes deployed [8] on a relatively large device. A recent paper from another commercial organization [16] used smaller Xilinx devices, utilizing only 2070 DSP48 blocks.

A brief survey of other recent works using our targeted Stratix 10 2800 device gives some additional datapoints. In [17], up to 100% DSP with 76% of the ALMs were used, showing that large designs are possible on this device with a variety of approaches, although this example only achieved

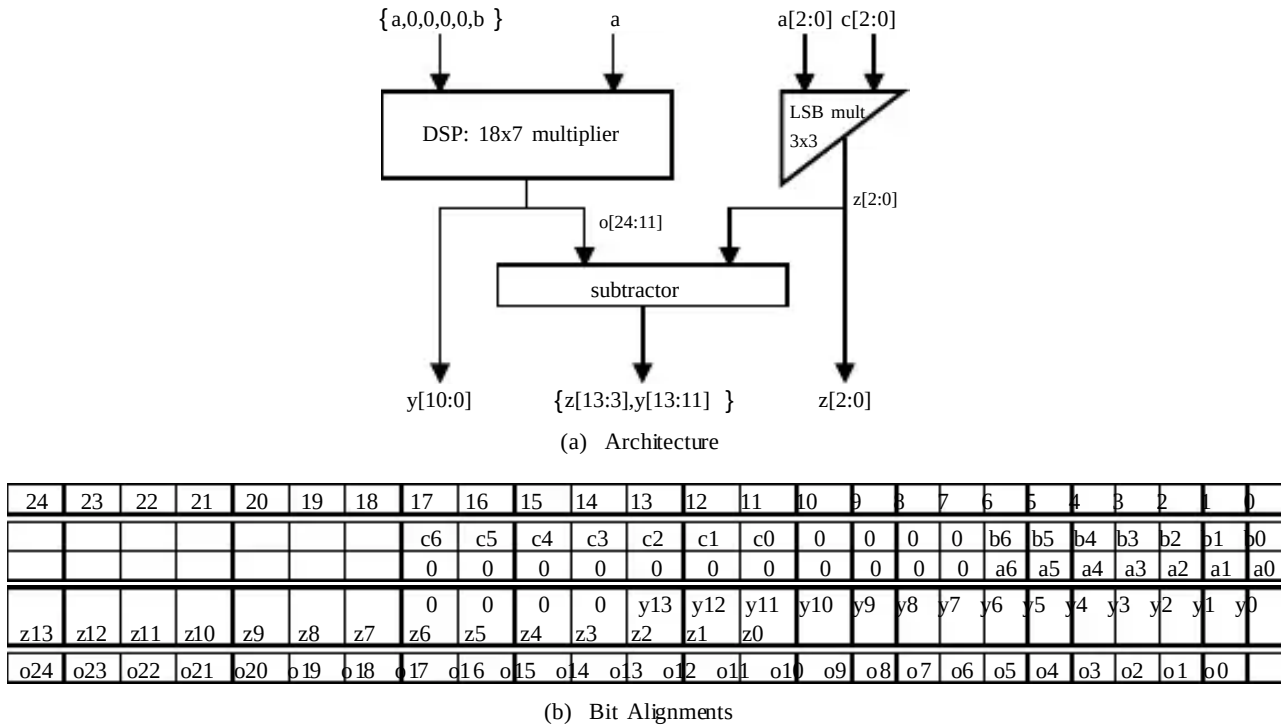


Fig. 1. DSP Block based unsigned 7x7 multiplier

240 MHz. Another design [18] had a larger ratio of DSP Blocks (71%) to ALMs (15%), but despite the relative sparseness of the utilization, was slower at 200 MHz.

Power was not reported for the VUP designs. Microsoft reports 125W for a full chip Stratix 10 2800, which uses the majority of resources at a lower frequency. The VUP designs use a significant amount of memory and are DSP rich, and have a relatively low logic utilization, but run the DSP blocks (and in [14], the memory as well) at very high speed to achieve throughput. While we cannot compare any Xilinx power numbers, we can see that the power envelope for a full device Stratix 10 2800 @400MHz should still be coolable in a production environment. We therefore set ourselves a target of a design that uses around 80% of the device resources (but, if possible, a lower percentage of memory blocks), capable of running in the range of 400 MHz on a previous generation FPGA. In [6], a relatively small VUP based CNN accelerator, using only 1,106 of the 6,840 DSP48 block on the device (although 79% of the logic) consumed 75W at a 200 MHz clock. This was apparently measured at the board level, so may not accurately reflect the FPGA power consumption. Power is reported for [17], but these are simulated numbers from the EPE tool, and not actually measured.

We will build number of dot product structures, balancing DSP based implementations with multipliers constructed from different ratios of M20K blocks and soft logic, in order to provide a wider choice in resource balancing. We will also arrange the dot products in a 2D systolic architecture so that memory bandwidth, or at least the I/O bandwidth is reduced. This may increase the stress on inter PE routing, but will also give a worst-case routing congestion datapoint. If a 2D systolic structure is not suitable, the dot products could still be individually instantiated. A key point of our designs over

[12][14] is that the dot product latency varies log rather than linearly of the dot size, reducing system latency, and giving more flexibility to the matrix and vector decompositions.

III. METHOD

We started by first creating some efficient SM1.7 operators, based on both DSP Blocks and M20K memory blocks. In both cases, a small amount of soft logic is required. Many of our dot products will also require pure soft logic multipliers, and we will use the Quartus based IP for these.

A. DSP Block Based Multipliers

In [15] several new DSP block based methods for INT8 multipliers were introduced. We will modify these to implement unsigned INT7 (UINT7) multipliers instead.

Both [15] and [11] use a signed or twos complement format to represent input values. There are some FPGA implementation efficiencies if we use SM format instead. The dynamic range is almost the identical, with the only difference that the number -128 cannot be represented in SM1.7. The sign is simply calculated as $(\text{sign}_a) \text{ XOR } (\text{sign}_b)$. Although the first level of the adder tree is arithmetically more complex as the signs can result in subtractions as well as addition, this can be supported in the logic associated with the FPGA carry chain, as is therefore free at the implementation level.

The magnitude bits require a UINT7 multiplier. We use a modified technique of [15], with each INT18 multiplier in the DSP block acting as an unsigned 18x7 multiplier. The two 7 bit multiplicands are input with zero nibble between them. This output of the multiplier contains the sum of two partially overlapping 14 bit results. A correction factor of 3 bits can be calculated by a subset multiplier, which only outputs the 3 LSBs of a 3x3 bit multiplication. The description in [15] does

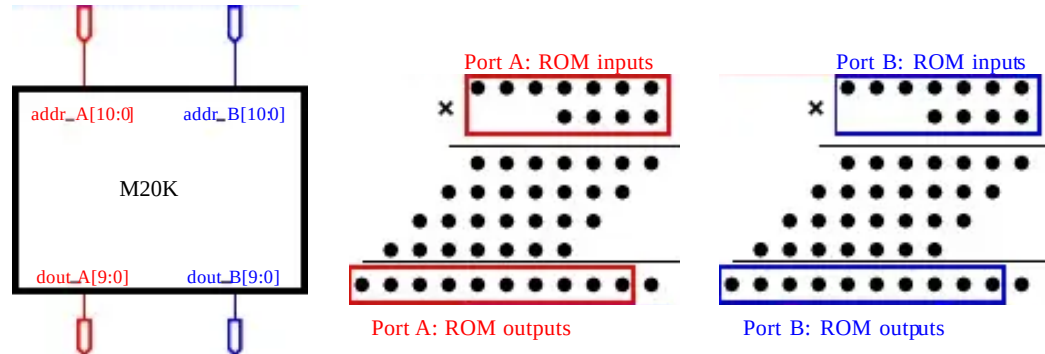


Fig. 2. M20K Dual Port ROM - Pair of 7x4 Multiplications

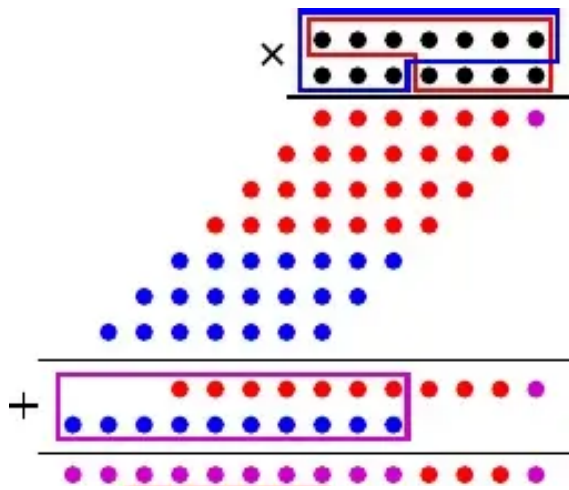


Fig. 3. Arithmetic Equivalence of M20K Mapping

not describe the construction of this multiplier, but the small size means that it can be implemented relatively efficiently by table look up, especially if 6LUTs are natively supported. We used the techniques in [19] to map this multiplier to the ALM structure in the Stratix 10 device.

We simply subtract the value $z[2:0]$ from $o[24:11]$ to obtain the upper 11 bits of $a*c$, or $z[13:3]$, and the 3 upper bits of $a*b$, or $y[13:11]$. The lower 3 bits of $a*c$ are output from the LSB-only multiplier, and the lower 11 bits of $a*b$ are output directly from the DSP Block. Both the LSB multiplier and the subtractor can be implemented in 7.5 ALMs, or just under 4 ALMs per UINT7 multiplier on average. Depending on the performance required, additional logic may be needed to introduce pipelining, although this may be amortized over larger structures like dot products.

We re-used the techniques of [15] for the INT8 dot products.

B. M20K Based Multipliers

The features and organization of the M20K memory block lends itself well to 7 bit multipliers. This 20Kb block can be configured as a dual port ROM, with an 11 bit address space, and therefore a 10 bit output. This supports two independent 7x4 multiplications, as shown in Figure 2.

One set of input multiplies $A[6:0]*B[3:0]$, and the other $A[6:0]*B[6:4]$ - in other words, we only use the 7x3 capability of the second input. As we only have 10 bits of the required

11 bit output available, we do not support the LSB, which can be inexpensively calculated in a 2 bit logic function. This is not an issue for the 7x3 case, as we treat the 3 bit input as a 4 bit value with a LSB, and right shift the output by 1 bit. From the dot diagram in Figure 2, the least significant partial product disappears, and we end up with 7x4 and 7x3 products, respectively. We then sum the appropriate segments to get our result, and append the separately calculated LSB. The total cost is 6.5 ALMs: 5 ALMs for the adder, 1/2 an ALM for the LSB, and 1.5 ALMs to latency balance the LSBs of the 7x4 multiplier. The latter can again be amortized across a dot product for additional savings.

We then extended the memory based multiplier method to INT8. Using the M20K here is more difficult. First of all, both multiplier halves in the UINT7 case were identical - the multiplicand was the same, and both multiplier subsets were unsigned, even though the upper one only used 3 of the 4 bit dataset. For INT8, the upper multiplier half is signed, while the lower half is unsigned (and identical to the UINT7 case). This means that the two multiplier nibbles need separate tables. To support this efficiently, a pair of INT8 multipliers need to be constructed together, with both upper and lower multiplier nibbles addressing their own dual port ROMs. The granularity of the INT8 M20K based method is therefore two.

Figure 4 shows a DOT representation of this. The blue and orange input boxes (and the same colored dots) represent the unsigned 7 bit multiplicand and signed 4 bit multipliers, and the red and green input boxes the unsigned 7 bit multiplicand and the unsigned 4 bit multiplier. The yellow dots on the LSBs of the partial products are calculated in soft logic, just like the LSB for the first partial product in the UINT7 case. The missing information is the product of the MSB of the multiplicand and the multiplier, also shown as yellow dots. Like the LSBs, these are calculated in soft logic. The output stage now has three rows, but we do not need to add them all here. The four MSBs of the third row can be moved to the empty positions of the first row; the existing partial product adder remains the same size. The remaining 4 bits in the third row can be added with the same 4 bits from any other multiplier in the DOT, and only folded into the final result at the output.

The INT8 version requires an additional 4.5 ALMs over the

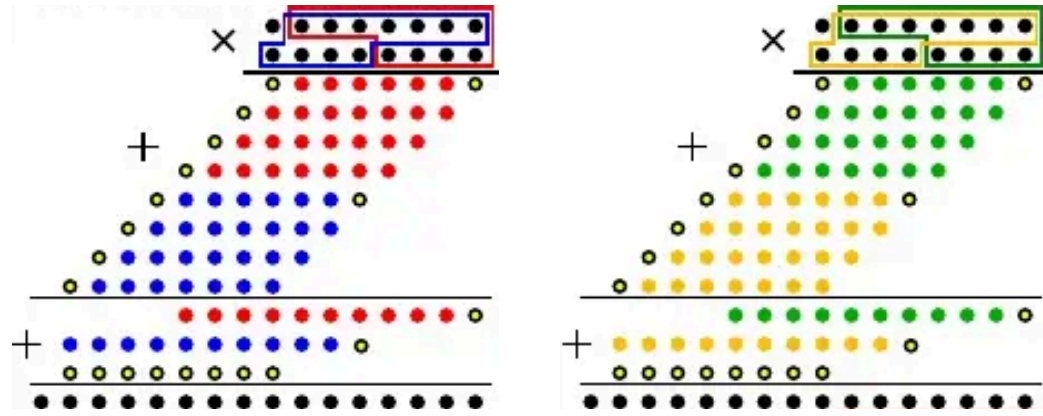


Fig. 4. Int8 M20K Mapping

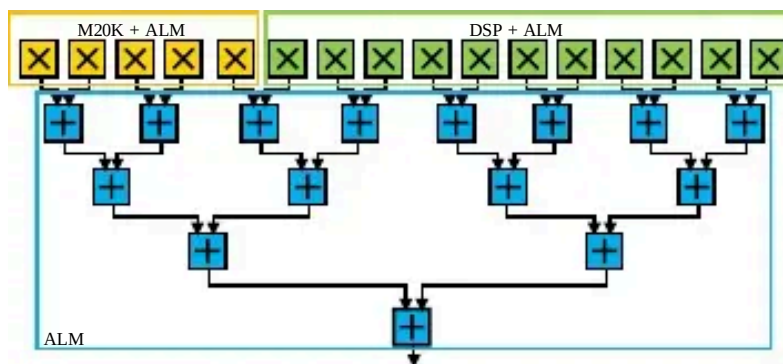


Fig. 5. Mixed Multiplier Construct 16 Element dot product

UINT7 counterpart. The total logic cost - around 10 ALMs - is considerably less than the 30 ALMs required for the pure soft logic IP provided by Quartus. The die cost of the M20K is likely considerably larger than the difference of 20 ALMs, but it does allow us additional choices in balancing resource use on the FPGA. As we are now attempting to fit multiple tens of thousands of multipliers on an FPGA, a small decrease in soft logic per multiplier can have a marked impact.

We can now assemble dot products. We use 16 element dot products to more closely correlate inter-PE routing resources with the density of long vertical wires in the Stratix 10 devices. (We originally tried 32 element dot products like [15], but found that the 2D systolic array architecture suffered from excessive routing congestion, affecting Fmax.) We will first implement SM1.7 based cores. Our 16 element dot products contain 11 DSP based SM1.7 multipliers, with a mix of M20K and logic based multipliers. Figure 5 shows a block diagram of a dot product with 5 M20K based multipliers, annotated with implementation resource per function (there are no purely logic based multipliers in this design).

We can then start creating systolic arrays of these dot products. Each PE consists of a 16 element dot product, and two directions of activation and weight routing, vertical and horizontal. Each PE also has a separate output. Our building block is a 2x2 PE grid, which we then replicate to make a larger systolic array. Figure 6 shows the connections per individual PE, and Figure 7 shows the logical structure of an

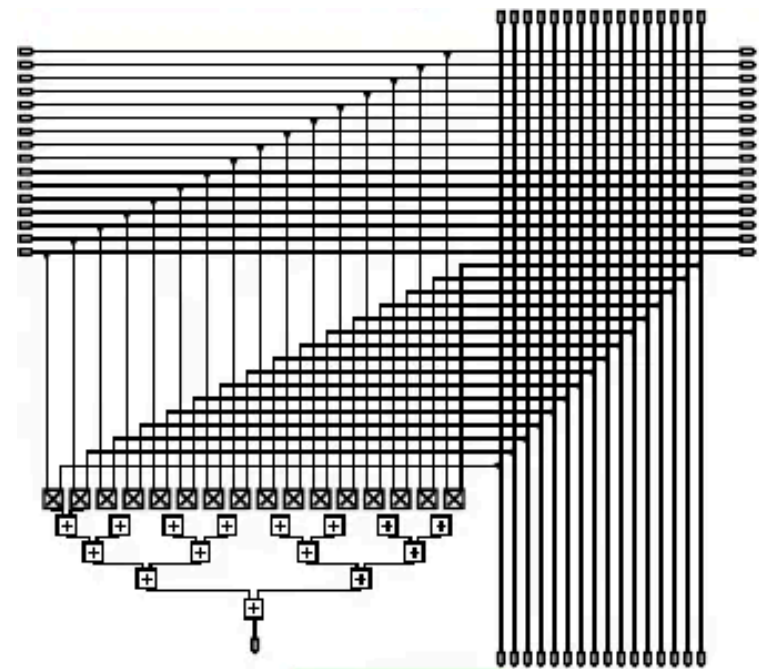


Fig. 6. PE Dot Product and Ingress/Egress Ports

TABLE I
RESOURCE REQUIREMENTS FOR DIFFERENT SM1.7 PE COMPONENT MIXES

M20Ks SM1.7	ALM (logic+FF)	ALM (logic)	ALM (FF only)	FFs	M20K	DSPs
0	1573	53	482	3694	0	11
1	1493	57	471	3512	4	11
2	1411	54	448	3330	8	11
3	1328	58	455	3148	12	11
4	1245	51	409	2966	16	11
5	1164	50	366	2784	20	11

array of PEs.

Table 1 shows the size of our base 2x2 PE grid, with varying numbers of M20K based SM1.7 multipliers per PE. In each case, 11 of the 16 multipliers in each PE are DSP based multipliers, with the remaining 5 being a combination of multipliers constructed from memory or soft logic. There is a significant cost to the logic based multipliers, but these may be required in cases where the system design requires a large number of memory ports to feed the dot products. We

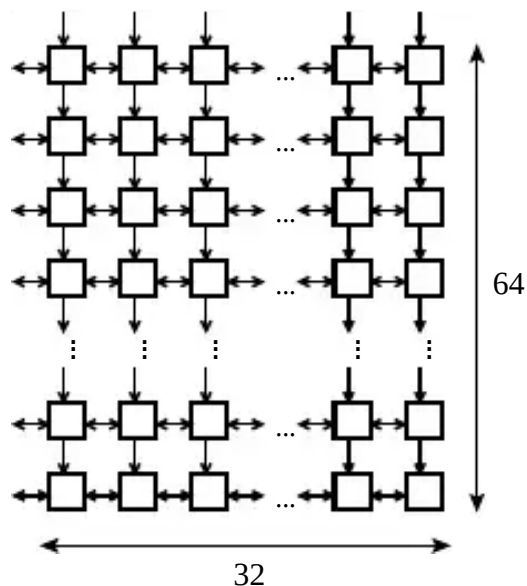


Fig. 7. 2D PE Array

TABLE II
RESOURCE REQUIREMENTS FOR AN EXAMPLE INT8 2x2 GRID

M20Ks INT8	ALM (logic+FF)	ALM (logic)	ALM (FF only)	FFs	M20K	DSPs
4	1606	54	730	4124	16	12

have seen in the earlier [8][12][14] cases that there is a large variation in the memory depth and bandwidth to multiplier relationship. Further implementation flexibility is possible by mixing grids with different multiplier composition ratios.

Because of the more restrictive memory based approach in the INT8 case, we only composed one version of the 16 element dot product, split into 4 and 12 elements of the M20K and DSP Block based versions, respectively. Table 2 reports the resources in the same manner as table 1.

For all of our designs, we targeted a 1SG280LN2F43E2VG, using Quartus 19.1. It is important to note that this is a mid-speed grade device.

IV. RESULTS - SYSTOLIC ARRAYS

We created a number of chip filling designs (to approximately 80% logic usage, as discussed earlier in our goals for this work), with different types of PE. In these reported cases, all PE compositions were uniform across the design, but this does not have to be the case. We first built these full chip designs with three variants of the SM1.7 dot products: 5, 3, and 2 M20K multipliers per PE. The results are listed in table 3. In every case, the number of PEs was 2048, arranged as a 32x64 grid of PEs (16x32 grid of our 2x2 grids). In all cases, these designs were placed and routed using a push button flow. No floorplanning was used, even for the very large arrays.

We can achieve a successful fit of 32K multipliers, with consistent results over different multiplier component mixes, while leaving enough logic left over to support a system design in the Brainwave style. Our performance consistently exceeds 400MHz, so a performance level of 25TOPs to 30TOPs is a

reasonable target for this device, and is likely scalable within this device family.

The INT8 versions gave us both slightly lower performance and lower density. We initially tried a near maximum DSP usage for a 30x60 grid of PEs (28,800 INT multipliers), but the speed dropped below 400 MHz. We attempted to improve the speed by pipelining the critical path, which was from the output of the DSP Blocks to the immediately following adders, but this only increased area, with a minimal impact on performance. Reducing the grid size was more successful - although the 25,088 multipliers in this grid is only a 12% density improvement on the previously reported Intel results, a large number of DSP Blocks have been freed up for other tasks, such as activation function implementation. About half the M20Ks remain for the system design. In all of our SM1.7 and INT8 designs, the dot product composition is homogeneous throughout the grid. By mixing and matching, e.g. having some PEs implemented only with DSP Blocks and some with mixture of DSP Blocks and memory, and possibly logic, different amounts and distributions of resources can remain free.

Our example systolic arrays used virtual pins, as in [15]. The number of virtual pins in our design was considerably less, because the 2D systolic structure includes PE to PE communication paths. There are a total of 51,200 virtual pins, consisting of 8K left side input bits, 4K bits in from the top, and 38,912 output bits. These virtual pins occupy 25600 ALMs.

A floorplan and routing heatmap are shown in Figure 8. This particular example was for SM1.7, with 5 M20K based multipliers per dot product. The heatmap for all of the SM1.7 and INT8 grids is almost identical, with the highest stress (colored in pink) concentrated around the discontinuities on the die. The include the embedded processor subsystem on the lower right edge, and the IO regions running vertically through the die. This suggests that a system using a collection of smaller dot product arrays, or cases where only a smaller portion of the FPGA is used for the deep learning application, may be able to increase the clock frequency of the dot product array significantly (we found that small grids often exceeded 600MHz).

Figure 9 shows an intermediate sized array (20x40 dot products), which is about 40% of the maximum size limit, based on both our resource goal, and the number of DSP Blocks in the device. The fractal synthesis clustering and packing algorithms directs Quartus to create an almost perfectly rectangular layout, leaving the majority of the device completely untouched.

The Brainwave design contained 96K multipliers with a 1.2 magnitude format. Our work shows that 32K multipliers for a much larger 1.7 magnitude format is readily achievable - this translates to about a 3x ISO density improvement, based on arithmetic complexity (which varies as the square of relative precision).

TABLE III
LARGE 2D SYSTOLIC ARRAY (SM1.7) RESULTS

M20Ks SM1.7	ALM (total)	ALM (logic+FF)	ALM (logic)	ALM (FF only)	FFs	M20K	DSPs	Max
5	654K (70%)	429K	24K	278K	1676K	10240 (87%)	5632 (98%)	432
3	735K (79%)	499K	23K	268K	1852K	6144 (52%)	5632 (98%)	445
2	774K (83%)	534K	22K	251K	1940K	4096 (35%)	5632 (98%)	413

TABLE IV
LARGE 2D SYSTOLIC ARRAY (INT8) RESULTS

Array Size	ALM (total)	ALM (logic+FF)	ALM (logic)	ALM (FF only)	FFs	M20K	DSPs	Max
30x60	727K (78%)	468K	20K	298K	1888K	7200 (61%)	5400 (94%)	378
28x56	693K (74%)	407K	18K	345K	1888K	6272 (54%)	4704 (82%)	404

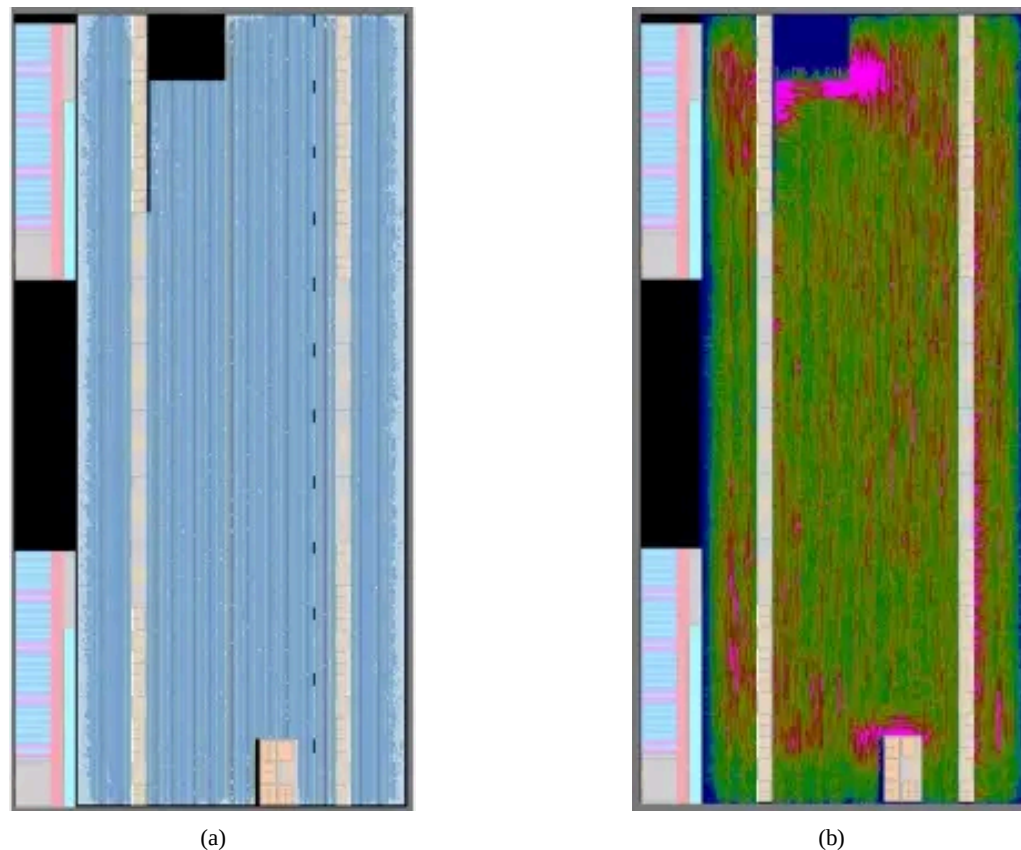


Fig. 8. Floorplan and Heatmap of a 32K multiplier 2D Systolic Array

V. RESULTS - SYSTEM DESIGNS

After validating our approach with the independent 2D systolic arrays, we implemented a more representative system design. In this case, all I/Os are mapped to pins. First, each PE in the systolic array is terminated in an 32b integer accumulator. The PE routing of figure 6 is modified to support systolic propagation of the accumulator results to the core periphery. Finally, small (512 word) input and output RAMs are attached to the periphery for flow control. Muxing is used to select the current output stream.

For any given array size, the number of DSP Blocks remains the same, while there are small (typically 5%) increases in soft logic and memory sizes. There is virtually no impact on speed. We can see that for even very large (30x60) PE arrays, there

is still some logic margin available, and for all array sizes, the memory availability is greater than the logic availability. All of this bodes well for incorporating these structures into a more complex deep learning system. Alternately, the system design can be used as an accelerator as is.

We noticed an initial speed degradation of about 10% compared to the array core, and decided to introduce some minimal floorplanning. Pinning the corners of the arrays achieved an immediate result, with the system designs running about 10% faster than the push button arrays.

Figure 10 shows a block diagram of the system array. The flow control memories can be seen in the periphery. The routing heatmap of Fig. 11 following our minimal floorplan intervention shows less stress than the systolic array core alone.

TABLE V
SYSTOLIC ARRAY SYSTEM (INT8) RESULTS

Array Size	ALM	Pins	FFs	M20K	DSPs	FMax (MHz)
24x48	503K (54%)	357 (39%)	1300K	4944 (42%)	3456 (60%)	464
26x52	588K (63%)	357 (39%)	1538K	5772 (49%)	4056 (70%)	458
28x56	681K (73%)	357 (39%)	1775K	6664 (57%)	4704 (82%)	439
30x60	778K (83%)	357 (39%)	2027K	7620 (65%)	5400 (94%)	402

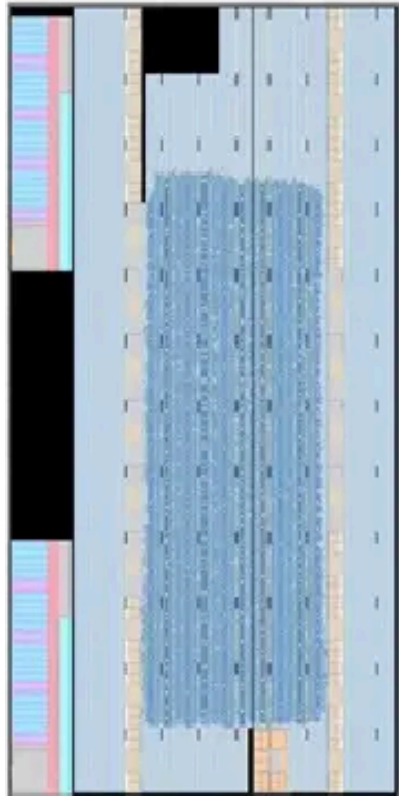


Fig. 9. 20x40 PE 2D Systolic Array

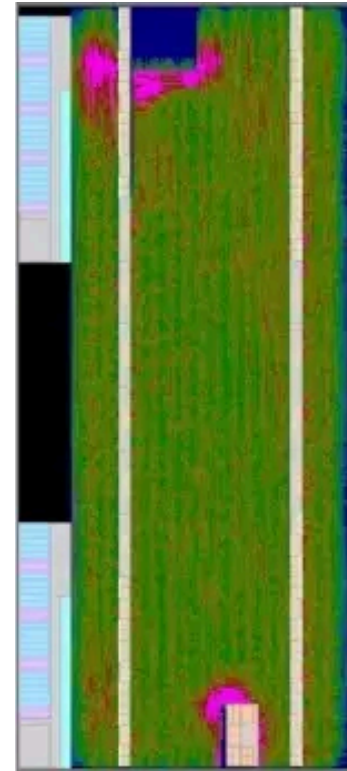


Fig. 11. Routing Heatmap

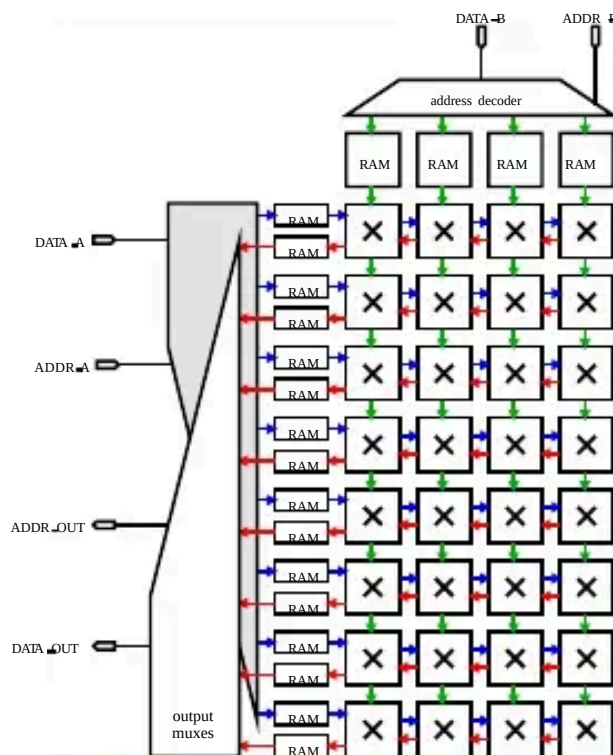


Fig. 10. Systolic Array Block Diagram

VI. CONCLUSIONS

Our goal was to develop a method to create large, scalable, and high performance systolic arrays for DL at an 8 bit precision. We showed new methods for INT8 multiplier based dot products, and introduced SM1.7 as an even more efficient format for implementing these arithmetic constructs with this precision. The resultant designs achieve their goals with consistent and deterministic speed, as well as low latency. Our throughput is 10% to 50% higher than other designs implemented on the same or slightly larger devices, by utilizing a balanced (the ratio of multipliers and soft logic to DSP blocks) implementation mix. We were able to realize this results even using a single monolithic chip scale 2D systolic array.

We can scale these arrays from small to large, while ensuring an efficiently packed, placed and routed result, without floorplanning, simplifying the design process greatly. We then extended the arrays to include flow control so that they can be used as an accelerator, either as a stand alone device, or as part of a more complex deep learning design on the FPGA. Using a small amount of floorplanning, performance increased, delivering over 400MHz on a mid-speed grade device, for even a very full design.

REFERENCES

- [1] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-I. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omemick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ser. ISCA '17. New York, NY, USA: ACM, 2017, pp. 1–12. [Online]. Available: <http://doi.acm.org/10.1145/3079856.3080246>
- [2] B. Pasca and M. Langhammer, "Activation function architectures for fpgas," in *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*, Aug 2018, pp. 43–437.
- [3] I. Swarbrick, D. Gaitonde, S. Ahmad, B. Gaide, and Y. Arbel, "Network-on-chip programmable platform in Versal™ ACAP architecture," in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '19. New York, NY, USA: ACM, 2019, pp. 212–221. [Online]. Available: <http://doi.acm.org/10.1145/3289602.3293908>
- [4] Intel Agilex Variable Precision DSP Blocks User Guide, 2019, https://www.intel.com/content/dam/altera-www/global/en_US/pdfs/literature/hb/agilex/ug-ag-dsp.pdf.
- [5] G. R. Chiu, A. C. Ling, D. Capalija, A. Bitar, and M. S. Abdelfattah, "Flexibility: Fpgas and cad in deep learning acceleration," in *Proceedings of the 2018 International Symposium on Physical Design*, ser. ISPD '18. New York, NY, USA: ACM, 2018, pp. 34–41. [Online]. Available: <http://doi.acm.org/10.1145/3177540.3177561>
- [6] H. Nakahara, Y. Sada, M. Shimoda, K. Sayama, A. Jinguji, and S. Sato, "FPGA-based training accelerator utilizing sparseness of convolutional neural network," in *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*, Sep 2019.
- [7] X. Lian, Z. Liu, Z. Song, J. Dai, W. Zhou, and X. Ji, "High-performance fpga-based cnn accelerator with block-floating-point arithmetic," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 8, pp. 1874–1885, Aug 2019.
- [8] J. Fowers, K. Ovtcharov, M. Papamichael, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, L. Adams, M. Ghandi, S. Heil, P. Patel, A. Sapek, G. Weisz, L. Woods, S. Lanka, S. K. Reinhardt, A. M. Caulfield, E. S. Chung, and D. Burger, "A configurable cloud-scale DNN processor for real-time AI," in *2018 ACM/IEEE 45th Annual*
- International Symposium on Computer Architecture (ISCA)*, June 2018, pp. 1–14.
- [9] E. Chung, J. Fowers, K. Ovtcharov, M. Papamichael, A. Caulfield, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman et al., "Accelerating persistent neural networks at datacenter scale," in *Hot Chips*, vol. 27, 2017.
- [10] M. Langhammer, G. Baeckler, and S. Gribok, "Fractal Synthesis: Invited tutorial," in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '19. New York, NY, USA: ACM, 2019, pp. 202–211. [Online]. Available: <http://doi.acm.org/10.1145/3289602.3293927>
- [11] Deep Learning with INT8 Optimization on Xilinx Devices, 2017, https://www.xilinx.com/support/documentation/white_papers/wp486-deep-learning-int8.pdf.
- [12] E. Wu, X. Zhang, D. Berman, I. Cho, and J. Thendean, "Compute-efficient neural-network acceleration," in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '19. New York, NY, USA: ACM, 2019, pp. 191–200. [Online]. Available: <http://doi.acm.org/10.1145/3289602.3293925>
- [13] E. Wu, X. Zhang, D. Berman, and I. Cho, "A high-throughput reconfigurable processing array for neural networks," in *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*, Sep. 2017, pp. 1–4.
- Machine Learning problems,* in *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*, Sep 2019.
- [15] M. Langhammer, B. Pasca, G. Baeckler, and S. Gribok, "Extracting int8 multipliers from int18 multipliers," in *International Conference on Field Programmable Logic and Applications*. Barcelona, Spain: IEEE, sep 2019.
- [16] D. Wu, Y. Zhang, X. Jia, L. Tian, T. Li, L. Sui, D. Xie, and Y. Shan, "A high-performance CNN processor based on FPGA for MobileNets," in *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*, Sep 2019.
- [17] S. K. Venkataramanaiah, Y. Ma, S. Yin, E. Nurvithadhi, A. Dasu, Y. Cao, and J.-S. Seo, "Automatic compiler based FPGA accelerator for CNN training," in *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*, Sep 2019.
- [18] R. Rajat, H. Zeng, and V. Prasanna, "A flexible design automation tool for accelerating quantized spectral CNNs," in *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*, Sep 2019.
- [19] M. Langhammer and G. Baeckler, "High density and performance multiplication for FPGA," in *25th IEEE Symposium on Computer Arithmetic, ARITH 2018*, Amherst, MA, USA, June 25–27, 2018, 2018, pp. 5–12. [Online]. Available: <https://doi.org/10.1109/ARITH.2018.8464695>
- [20] X. Yu, Y. Wang, J. Miao, E. Wu, H. Zhang, Y. Meng, B. Zhang, B. Min, D. Chen, and J. Gao, "A data-center FPGA acceleration platform for convolutional neural networks," in *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*, Sep 2019.
- [14] A. Samajdar, T. Garg, T. Krishna, and N. Kapre, "Scaling the Cascades: Interconnect-aware Mapping Strategies for FPGA implementation of